AI_GPU_EVENG_LABS · DGX_CLUSTER_TRAINING_JOB_LAB · JUNIPER JUNOS EDITION

DGX Spark 4-Node RoCE Cluster Bring-Up and Validation Lab Guide

From the first cable to a GPU-to-GPU training job: physical connections, topology, IP plan, switch and host configuration, power-on, bootstrap, validation at every layer, and performance tests, with the commands and the output to expect at each step.

Compute	4 × NVIDIA DGX Spark (GB10 Grace Blackwell, 1 GPU and 128 GB unified memory per node), DGX OS (Ubuntu 24.04 based)
Fabric	2 × Juniper PTX10002-36QDD, joined by a 2 × 400G LACP inter-switch link (ae0). One 200G QSFP56 link per Spark. RoCEv2, MTU 9216, one L2 domain.
Management	Spark 10GbE RJ45 and switch `em0` on a separate 1/10GbE management switch
Software	NVIDIA driver 580 / CUDA 13.0, NCCL 2.29 + nccl-tests, Open MPI, perftest, PyTorch 2.12 (cu130)
Tests	Link, L2/L3, jumbo frames, RDMA devices and GIDs, raw RDMA bandwidth (pair, both halves, full mesh), GPU checks, GPU-to-GPU point-to-point, NCCL collectives on 2 and 4 nodes, PyTorch all-reduce, multi-node DDP training

Contents

1	Lab at a glance: physical topology and bill of materials
2	Inside a DGX Spark: why one 200G port shows up as two interfaces
3	Cabling plan (cutsheet)
4	Logical topology and IP plan
5	Bring-up sequence overview
6	Phase 1 · Physical install
7	Phase 2 · Switch configuration
8	Phase 3 · Power on and first boot
9	Phase 4 · Host fabric configuration
10	Phase 5 · Cluster bootstrap: SSH, MPI, NCCL, PyTorch
11	Phase 6 · Validation ladder
12	Phase 7 · Performance tests: RDMA and GPU-to-GPU
13	Phase 8 · Workloads: distributed check and DDP training
14	Troubleshooting
A–D	Environment reference · Results record sheet · Corrections to the repo README · Sign-off checklist

Prepared for Mohammad Tahir · October 2026 · Based on the repo's 4-node testbed and NVIDIA's DGX Spark playbooks. Example values (management IPs, switch port numbers, QoS profile names) are marked; substitute your own.

1 · Lab at a glance

Four DGX Sparks connect to two Juniper PTX switches, two Sparks per switch. Each Spark uses one 200G QSFP56 port (Port 0). Each switch provides those links from one 400G QSFP-DD port split into two 200G legs with a breakout cable. The two switches are joined by two 400G links bundled as `ae0`. A separate management network carries SSH, setup and switch management.

Physical topology: one 200G link per Spark, 2 × 400G between switches

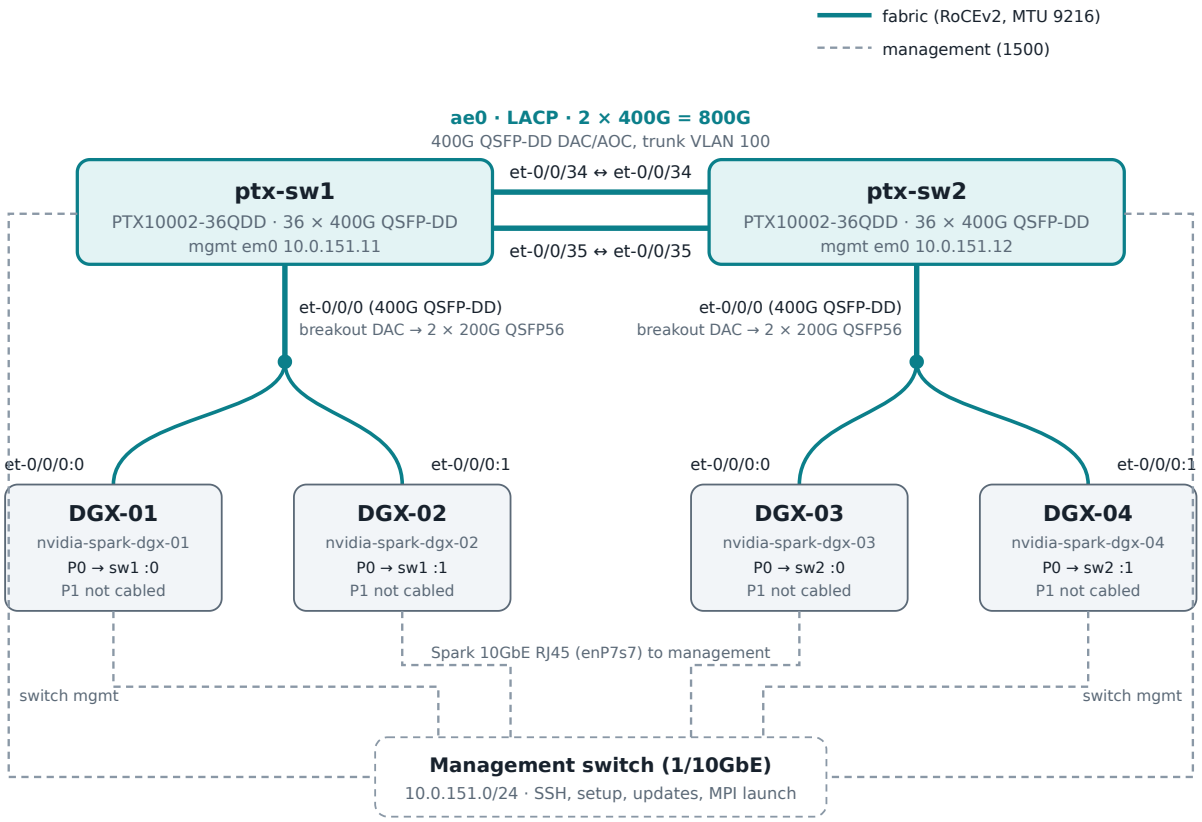


Figure 1. Physical topology. Switch port numbers, breakout cabling and management addresses are example values; match them to your switch and record them in the cutsheet (section 3).

Bill of materials

Item	Qty	Purpose / notes
NVIDIA DGX Spark (GB10), with its USB-C power adapter	4	Compute nodes. One GPU each, 128 GB LPDDR5x unified memory, ConnectX-7 with 2 × QSFP 200G, 10GbE RJ45.
Juniper PTX10002-36QDD	2	Fabric switches (the repo's testbed). Any 200G-capable switch with lossless Ethernet works the same way.
400G QSFP-DD to 2 × 200G QSFP56 breakout DAC (≤ 3 m)	2	One per switch; each serves two Sparks. Alternative: one 200G port and a straight QSFP56 cable per Spark.
400G QSFP-DD DAC or AOC	2	Inter-switch link members (et-0/0/34, et-0/0/35).
Management switch, 1 or 10GbE, plus Cat6 patch cords	1 + 6	4 × Spark RJ45, 2 × switch `em0`. Needs a DHCP server or static addressing and internet access for updates.
Rack shelf, PDU outlets, labels	—	Sparks are desktop units: shelf-mount, keep front and rear airflow clear.

Bandwidth budget

Each side of the fabric has 2 Sparks × 200G = 400G of host bandwidth, and the ISL carries 800G, so traffic between switches is not oversubscribed. In practice each Spark's PCIe path to its NIC limits it to roughly 2 × 100 Gb/s (section 2), well under the ISL.

2 · Inside a DGX Spark

The GB10 superchip joins a 20-core Grace CPU and a Blackwell GPU over NVLink-C2C, and both share 128 GB of unified memory: there is no separate GPU memory and no copy over PCIe. The ConnectX-7 NIC hangs off the GB10 through **two PCIe Gen5 x4 links**, in PCI domains 0000 and 0002. Linux therefore sees every physical QSFP port **twice**, once per PCIe link. That is why one cable produces two interfaces and two RDMA devices.

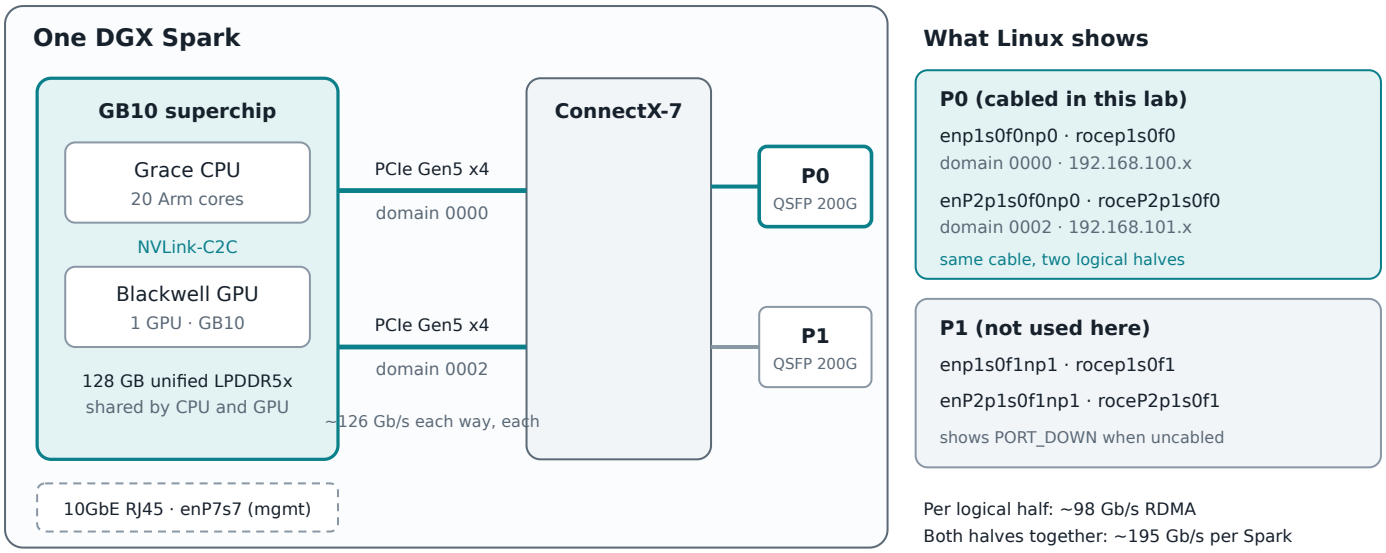


Figure 2. Internal connectivity. Each physical port appears as two Linux interfaces and two RDMA devices because the NIC sits on two PCIe Gen5 x4 links. Port positions: NVIDIA's playbook describes Port 1 as the QSFP cage further from the RJ45 port.

What this means for the lab

Observation	Explanation
`ib_write_bw` on one device tops out near 98 Gb/s	A single logical half is limited by its PCIe Gen5 x4 link (~126 Gb/s raw), not by the 200G port.
NCCL busbw near 24 GB/s	NCCL drives both halves (`rocep1s0f0` + `roceP2p1s0f0`): about 2 × 98 Gb/s ≈ 24 GB/s.
Two subnets on one cable	Each half gets its own subnet (100.x and 101.x) so the kernel and NCCL never see two routes to the same network. NVIDIA's playbook requires this.
Interfaces on Port 1 show Down	Expected when Port 1 is not cabled. Use the same port on every Spark.

Repo note

The repo's IP plan already uses Port 0 (both halves). Its training guide sets `NCCL_IB_HCA=rocep1s0f1`, which is Port 1. In this guide all NCCL and PyTorch runs use `rocep1s0f0,roceP2p1s0f0` so traffic stays on the cabled, addressed port.

3 · Cabling plan (cutsheet)

Build and review this before racking. Every cable gets a label at both ends that matches the **Label** column. Port numbers are examples; what matters is that every row is filled in and checked.

Label	A end	A port	B end	B port	Cable / optic	Speed	Network
F-01	ptx-sw1	et-0/0/0:0	nvidia-spark-dgx-01	QSFP P0	400G→2×200G breakout DAC, leg 0	200G	Fabric
F-02	ptx-sw1	et-0/0/0:1	nvidia-spark-dgx-02	QSFP P0	same breakout, leg 1	200G	Fabric
F-03	ptx-sw2	et-0/0/0:0	nvidia-spark-dgx-03	QSFP P0	400G→2×200G breakout DAC, leg 0	200G	Fabric
F-04	ptx-sw2	et-0/0/0:1	nvidia-spark-dgx-04	QSFP P0	same breakout, leg 1	200G	Fabric
I-01	ptx-sw1	et-0/0/34	ptx-sw2	et-0/0/34	400G QSFP-DD DAC/AOC	400G	ISL ae0
I-02	ptx-sw1	et-0/0/35	ptx-sw2	et-0/0/35	400G QSFP-DD DAC/AOC	400G	ISL ae0
M-01..04	dgx-01..04	10GbE RJ45	mgmt switch	ports 1–4	Cat6	10G	Mgmt
M-05, 06	ptx-sw1, sw2	em0	mgmt switch	ports 5–6	Cat6	1G	Mgmt
P-01..04	dgx-01..04	USB-C power	PDU A/B	outlets	vendor adapter	—	Power

Interface map (fill from the cutsheet, then verify in Phase 6)

Node	Switch port	Netdev (P0, dom 0000)	RDMA dev	Netdev (P0, dom 0002)	RDMA dev	MAC (P0 half 0)
dgx-01	sw1 et-0/0/0:0	enp1s0f0np0	rocep1s0f0	enP2p1s0f0np0	roceP2p1s0f0	read with `ip link`
dgx-02	sw1 et-0/0/0:1	enp1s0f0np0	rocep1s0f0	enP2p1s0f0np0	roceP2p1s0f0	
dgx-03	sw2 et-0/0/0:0	enp1s0f0np0	rocep1s0f0	enP2p1s0f0np0	roceP2p1s0f0	
dgx-04	sw2 et-0/0/0:1	enp1s0f0np0	rocep1s0f0	enP2p1s0f0np0	roceP2p1s0f0	

Physical rules

- Use the **same QSFP port (P0)** on every Spark. Mixing ports breaks the identical-interface assumption in every script.
- Keep DAC runs short (≤ 3 m for 200G/400G passive copper); use AOC between racks.
- Respect bend radius; don't let breakout cables pull on the Spark's QSFP cage.
- Sparks pull air front to back: no blocked vents, nothing stacked on top unless you use NVIDIA's stacking guidance.
- Split power across two PDUs where possible, so one PDU failure takes out only half the nodes.

4 · Logical topology and IP plan

The fabric is one Layer 2 domain (VLAN 100, `spark-fabric`) stretched across both switches over `ae0`. Two IP subnets ride on it, one per logical half of P0. Management is a separate network with its own subnet and default gateway; the fabric has **no** default gateway.

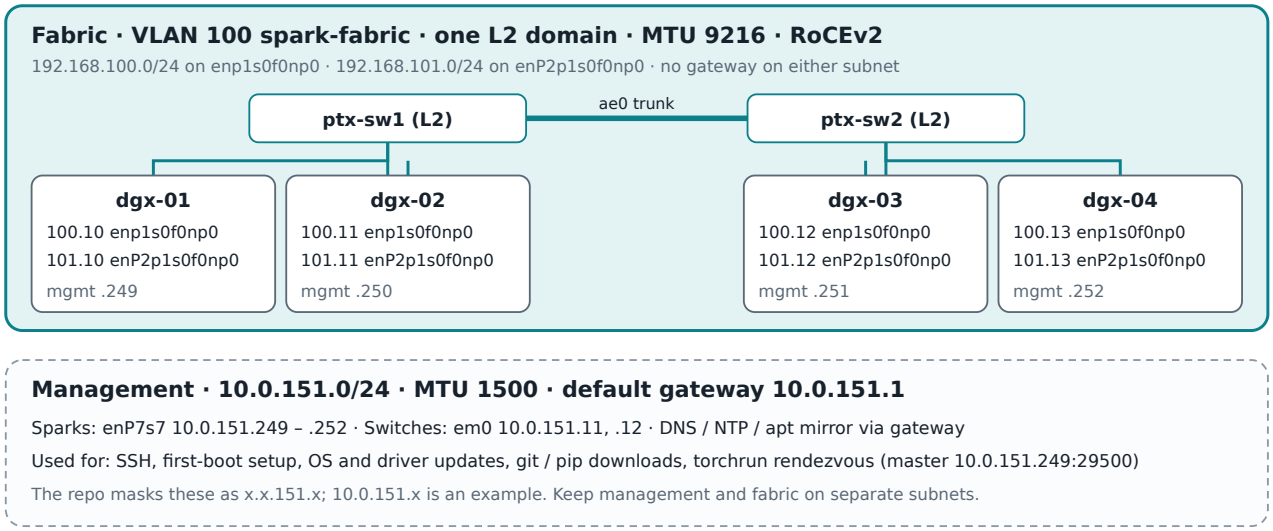


Figure 3. Logical topology. Fabric addresses are from the repo; management addresses are examples.

Address plan

Host	Mgmt (enP7s7)	enp1s0f0np0	enP2p1s0f0np0	RoCEv2 GID idx 3 (half 0)	RoCEv2 GID idx 3 (half 1)
nvidia-spark-dgx-01	10.0.151.249/24	192.168.100.10/24	192.168.101.10/24	`::ffff:c0a8:640a`	`::ffff:c0a8:650a`
nvidia-spark-dgx-02	10.0.151.250/24	192.168.100.11/24	192.168.101.11/24	`::ffff:c0a8:640b`	`::ffff:c0a8:650b`
nvidia-spark-dgx-03	10.0.151.251/24	192.168.100.12/24	192.168.101.12/24	`::ffff:c0a8:640c`	`::ffff:c0a8:650c`
nvidia-spark-dgx-04	10.0.151.252/24	192.168.100.13/24	192.168.101.13/24	`::ffff:c0a8:640d`	`::ffff:c0a8:650d`
ptx-sw1 / ptx-sw2	10.0.151.11 / .12	L2 only on the fabric (no IRB needed)			

GID index 3 is the RoCE v2 entry that carries the interface's IPv4 address (indexes 0 and 1 are link-local v1/v2, index 2 is IPv4 v1). It exists only after the interface has an IPv4 address.

Naming and conventions

Item	Convention
Hostnames	`nvidia-spark-dgx-01` … `-04`; add all four to `/etc/hosts` on each node with their management IPs.
User account	The same username on every node (the repo uses `regress`). MPI and the discovery script assume it.
Rank order	Rank 0 = dgx-01 (also the torchrun master), then 02, 03, 04.
MTU	9216 on Spark fabric interfaces and switch ports; RDMA then negotiates 4096-byte frames.

Lossless class	RoCE on DSCP 26 / priority 3 with PFC; NCCL traffic class 106 (= DSCP 26 with ECN-capable bits). Optional in a small lab, recommended under load.

5 · Bring-up sequence

Work in this order. Each phase ends with an exit check; don't start the next phase until it passes, because a fault found early is one cable or one setting, and the same fault found during an NCCL test looks like a slow GPU.

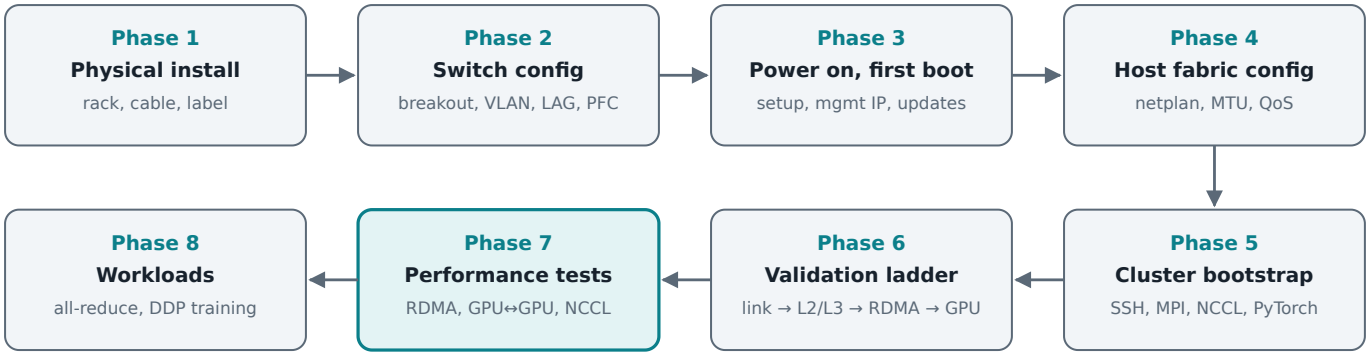


Figure 4. Bring-up phases. Phase 2 can run in parallel with Phase 3 once the switches have power.

Phase	Exit check (must pass)	Where
1 Physical	Every cutsheet row installed, labelled, visually audited	Rack
2 Switch	Breakout legs up at 200G, ae0 collecting/distributing, LLDP sees both peers, VLAN 100 on all fabric ports, MTU 9216	ptx-sw1, ptx-sw2
3 Power on	All 4 nodes reachable by SSH on management; same DGX OS, driver, CUDA and NIC firmware on all	Each node
4 Host fabric	Correct IPs on both halves, MTU 9216, link speed 200000Mb/s	Each node
5 Bootstrap	Passwordless SSH both ways; `mpirun` hostname on 4 ranks; NCCL and nccl-tests built on all nodes; torch sees the GPU	dgx-01 + all
6 Validation	Jumbo ping clean on every pair; GID index 3 present; RDMA active_mtu 4096; GPU healthy	All
7 Performance	~98 Gb/s per half, ~195 Gb/s both halves; NCCL busbw within 10% of baseline; NCCL log shows NET/IB	dgx-01 launches
8 Workloads	All-reduce returns 6.0 on 4 ranks; DDP completes and saves a checkpoint	All four

6 · Phase 1 · Physical install

1. **Place the switches** and connect both PSUs of each switch to separate PDUs. Don't power on yet.
2. **Place the Sparks** on a shelf with clear front and rear airflow. Connect each USB-C power adapter but leave the node off.
3. **Management first.** Cable each Spark's 10GbE RJ45 and each switch's `em0` to the management switch (M-01 to M-06). This is how you reach anything that will not come up on the fabric.
4. **Fabric.** Plug the 400G end of each breakout DAC into `et-0/0/0` on its switch. Plug leg `:0` into P0 of the first Spark and leg `:1` into P0 of the second (F-01 to F-04). Push until the latch clicks; pull gently to confirm.
5. **ISL.** Connect `et-0/0/34` ↔ `et-0/0/34` and `et-0/0/35` ↔ `et-0/0/35` (I-01, I-02).
6. **Label** both ends of every cable with its cutsheet label.
7. **Audit** against the cutsheet: right port, right cable type, bend radius, nothing pulling on a cage.

Exit check
Every cutsheet row is installed and labelled, and a second person has read the labels back against the sheet. The electronic cabling check (LLDP and link state) happens in Phase 2.

7 · Phase 2 · Switch configuration

Power on both switches and connect to the console or to `em0`. The configuration below is for ptx-sw1; ptx-sw2 is identical except for host-name, `em0` address and the descriptions.

Example configuration
Syntax is Junos OS Evolved style and is an example. Port channelization, Layer 2 bridging and lossless QoS options differ by platform and release: check the PTX10002-36QDD documentation for your release, especially breakout naming (`et-0/0/0:0`) and ethernet-switching support. On a platform without L2 bridging, use EVPN-VXLAN or a routed design instead.

PTX-SW1 · BASE, MANAGEMENT, LLDP

```
set system host-name ptx-sw1
set interfaces em0 unit 0 family inet address 10.0.151.11/24
set routing-options static route 0.0.0.0/0 next-hop 10.0.151.1
set protocols lldp interface all
```

BREAKOUT 400G INTO 2 × 200G TOWARD THE SPARKS

```
set interfaces et-0/0/0 number-of-sub-ports 2
set interfaces et-0/0/0 speed 200g
```

FABRIC VLAN AND SPARK-FACING PORTS (MTU 9216, FIXED 200G)

```
set vlans spark-fabric vlan-id 100
set interfaces et-0/0/0:0 description "nvidia-spark-dgx-01 P0"
set interfaces et-0/0/0:0 mtu 9216
set interfaces et-0/0/0:0 unit 0 family ethernet-switching interface-mode access
set interfaces et-0/0/0:0 unit 0 family ethernet-switching vlan members spark-fabric
set interfaces et-0/0/0:1 description "nvidia-spark-dgx-02 P0"
set interfaces et-0/0/0:1 mtu 9216
set interfaces et-0/0/0:1 unit 0 family ethernet-switching interface-mode access
set interfaces et-0/0/0:1 unit 0 family ethernet-switching vlan members spark-fabric
```

INTER-SWITCH LINK AE0 (LACP, TRUNK)

```
set chassis aggregated-devices ethernet device-count 4
set interfaces et-0/0/34 ether-options 802.3ad ae0
set interfaces et-0/0/35 ether-options 802.3ad ae0
set interfaces ae0 description "ISL to ptx-sw2"
set interfaces ae0 mtu 9216
set interfaces ae0 aggregated-ether-options lacp active
set interfaces ae0 aggregated-ether-options lacp periodic fast
set interfaces ae0 unit 0 family ethernet-switching interface-mode trunk
set interfaces ae0 unit 0 family ethernet-switching vlan members spark-fabric
```

LOSSLESS ROCE: PFC ON PRIORITY 3, DSCP 26 CLASSIFIER (RECOMMENDED; PROFILE NAMES ARE EXAMPLES)

```
set class-of-service forwarding-classes class lossless queue-num 3 no-loss
set class-of-service classifiers dscp rocev2-dscp forwarding-class lossless loss-priority low code-points 011010
set class-of-service congestion-notification-profile rocev2-pfc input ieee-802.1 code-point 011 pfc
set class-of-service interfaces et-0/0/0:0 classifiers dscp rocev2-dscp
set class-of-service interfaces et-0/0/0:0 congestion-notification-profile rocev2-pfc
set class-of-service interfaces et-0/0/0:1 classifiers dscp rocev2-dscp
set class-of-service interfaces et-0/0/0:1 congestion-notification-profile rocev2-pfc
set class-of-service interfaces ae0 classifiers dscp rocev2-dscp
set class-of-service interfaces ae0 congestion-notification-profile rocev2-pfc
# add ECN marking on queue 3 per the platform's RoCE / lossless guide
commit
```

Verify the switches

Command	Pass when
`show interfaces terse | match "et-0/0/0|ae0"`	`et-0/0/0:0`, `:1` and `ae0` are up/up (Spark legs come up only once the Sparks are powered)
`show interfaces et-0/0/0:0 | match "Speed|MTU"`	Speed 200Gbps, MTU 9216
`show lacp interfaces ae0`	Both members: Current, Collecting distributing
`show lldp neighbors`	et-0/0/34 and et-0/0/35 see the other switch; Spark legs list the Spark (needs `lldpd` on the Sparks)
`show ethernet-switching table vlan-name spark-fabric`	After Phase 4: 8 MACs (2 per Spark), local on et-0/0/0:x, remote on ae0.0
`show class-of-service interface et-0/0/0:0`	Congestion-notification Enabled, classifier rocev2-dscp

```
lab@ptx-sw1> show lacp interfaces ae0
Aggregated interface: ae0
    LACP state:       Role   Exp   Def  Dist  Col  Syn  Aggr  Timeout  Activity
      et-0/0/34      Actor    No    No   Yes  Yes  Yes   Yes     Fast    Active
      et-0/0/34    Partner    No    No   Yes  Yes  Yes   Yes     Fast    Active
      et-0/0/35      Actor    No    No   Yes  Yes  Yes   Yes     Fast    Active
      et-0/0/35    Partner    No    No   Yes  Yes  Yes   Yes     Fast    Active
    LACP protocol:        Receive State  Transmit State          Mux State
      et-0/0/34                Current   Fast periodic Collecting distributing
      et-0/0/35                Current   Fast periodic Collecting distributing
```

Exit check

ae0 is up with both members distributing; LLDP sees the peer switch on both members; VLAN 100 is on every Spark leg and on ae0; MTU 9216 everywhere on the fabric.

8 · Phase 3 · Power on and first boot

1. **Power on one Spark at a time** with its power button. A cold first boot takes a few minutes.
2. **Run first-boot setup.** Use a local display and keyboard, or the setup Wi-Fi hotspot printed on the Spark's quick-start card. Set language and time zone, create the **same username on every node** (the repo uses `regress`), and set the hostname `nvidia-spark-dgx-0N`.
3. **Management IP.** Give `enP7s7` its address from section 4, as a DHCP reservation or a static netplan entry, with gateway 10.0.151.1 and DNS. Then SSH in from your workstation: `ssh regress@10.0.151.249`.
4. **Update to one software baseline** on every node and reboot:

```
sudo apt update && sudo apt full-upgrade -y
sudo reboot
```

5. **Record versions**. They must match on all four nodes:

```
cat /etc/dgx-release | grep -E "DGX_OTA_VERSION|DGX_SWBUILD_VERSION"
nvidia-smi --query-gpu=driver_version --format=csv,noheader
nvcc --version | tail -2
ethtool -i enp1s0f0np0 | grep -E "driver|firmware-version"
uname -r
```

6. **Install the lab tools** (some may already be present on DGX OS):

```
sudo apt install -y openmpi-bin libopenmpi-dev perftest infiniband-diags rdma-core \
                    ibverbs-utils lldpd avahi-utils git build-essential python3-venv
sudo systemctl enable --now lldpd
```

7. **Name resolution.** On every node, add all four nodes to `/etc/hosts`:

```
10.0.151.249  nvidia-spark-dgx-01
10.0.151.250  nvidia-spark-dgx-02
10.0.151.251  nvidia-spark-dgx-03
10.0.151.252  nvidia-spark-dgx-04
```

Item	dgx-01	dgx-02	dgx-03	dgx-04
DGX OS / kernel				
Driver (expect 580.x)				
CUDA (expect 13.0)				
CX-7 firmware				

Exit check
All four nodes reachable by SSH on management; the version table has identical values in every column. Mixed versions are a top cause of NCCL hangs later.

9 · Phase 4 · Host fabric configuration

Write one netplan file per node for the two halves of P0, then apply. NVIDIA's playbook also restricts the file to root (`chmod 600`).

DGX-01 **/ETC/NETPLAN/40-CX7.YAML**

```
sudo tee /etc/netplan/40-cx7.yaml >/dev/null <<'EOF'
network:
  version: 2
  ethernets:
    enp1s0f0np0:
      addresses:
        - 192.168.100.10/24
      dhcp4: no
      mtu: 9216
    enP2p1s0f0np0:
      addresses:
        - 192.168.101.10/24
      dhcp4: no
      mtu: 9216
EOF
sudo chmod 600 /etc/netplan/40-cx7.yaml
sudo netplan generate
sudo netplan apply
```

Repeat on dgx-02, 03, 04 changing only the last octet (.11, .12, .13) on **both** lines. The interface names are identical on every node.

Watch for the repo typo

The repo's DGX-04 block uses `enP2p1s0f1np1` and `192.168.101.23`. Use `enP2p1s0f0np0` and `192.168.101.13`, as above.

Verify on each node

```
ip -4 -br a
ip link show | grep -E "enp1s0f0np0|enP2p1s0f0np0" | grep -o "mtu [0-9]*"
sudo ethtool enp1s0f0np0 | grep -E "Speed|Link detected"
ibdev2netdev
```

```
lo               UNKNOWN        127.0.0.1/8
enP7s7           UP             10.0.151.249/24
enp1s0f0np0      UP             192.168.100.10/24
enP2p1s0f0np0    UP             192.168.101.10/24
mtu 9216
mtu 9216
        Speed: 200000Mb/s
        Link detected: yes
rocep1s0f0 port 1 ==> enp1s0f0np0 (Up)
rocep1s0f1 port 1 ==> enp1s0f1np1 (Down)
roceP2p1s0f0 port 1 ==> enP2p1s0f0np0 (Up)
roceP2p1s0f1 port 1 ==> enP2p1s0f1np1 (Down)
```

If the speed shows 100000Mb/s

Auto-negotiation settled low. NVIDIA's playbook fixes this on the switch: disable auto-negotiation on the Spark leg and set the speed to 200G (in this design, `speed 200g` is already fixed on the breakout parent). Re-check with `ethtool`.

Optional: host lossless settings (only if the switches run PFC)

```
sudo mlnx_qos -i enp1s0f0np0   --trust dscp --pfc 0,0,0,1,0,0,0,0
sudo mlnx_qos -i enP2p1s0f0np0 --trust dscp --pfc 0,0,0,1,0,0,0,0
# NCCL then marks RoCE traffic DSCP 26 with ECN-capable bits:
export NCCL_IB_TC=106
```

`mlnx_qos` ships with the NVIDIA networking tools; these settings don't survive a reboot unless you persist them (for example with a systemd unit). Match the host priority to the switch's PFC priority, or PFC does nothing.

Exit check

Every node shows both P0 halves Up with the expected IP, MTU 9216 and 200000Mb/s. Port 1 devices show Down.

10 · Phase 5 · Cluster bootstrap

5.1 Discovery and passwordless SSH

NVIDIA's `discover-sparks` script finds the other nodes over the fabric with Avahi, creates one shared key and installs it everywhere. Run it on dgx-01 as the normal user (not root).

```
DGX-01 wget https://raw.githubusercontent.com/NVIDIA/dgx-spark-playbooks/main/nvidia/connect-two-sparks/assets/discover-sparks
chmod +x discover-sparks
./discover-sparks
```

```
Found: 192.168.100.11 (nvidia-spark-dgx-02.local)
Found: 192.168.100.12 (nvidia-spark-dgx-03.local)
Found: 192.168.100.13 (nvidia-spark-dgx-04.local)
Found: 192.168.101.11 (nvidia-spark-dgx-02.local)
...
Shared SSH setup complete!
```

Fallback without Avahi: `ssh-keygen -t ed25519` then `ssh-copy-id regress@192.168.100.1X` for each peer. Test every node, by fabric IP, from dgx-01:

```
for ip in 192.168.100.10 192.168.100.11 192.168.100.12 192.168.100.13; do ssh $ip hostname; done
```

5.2 MPI launch check

```
DGX-01 mpirun -np 4 \
  -H 192.168.100.10:1,192.168.100.11:1,192.168.100.12:1,192.168.100.13:1 \
  --mca plm_rsh_agent "ssh -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no -x" \
  hostname
```

```
nvidia-spark-dgx-01
nvidia-spark-dgx-03
nvidia-spark-dgx-02
nvidia-spark-dgx-04
```

Order varies. An "ORTE was unable to reliably start one or more daemons" error means SSH or fabric reachability to one node is broken.

5.3 Build NCCL and nccl-tests (on every node)

There is no shared storage, and `mpirun` needs the same binary at the same path on every rank, so build on all four. Clone both repos in the home directory (the repo README clones nccl-tests inside `~/nccl`, which does not match its run commands).

```
ALL NODES cd ~
git clone https://github.com/NVIDIA/nccl.git
cd nccl
make -j src.build NVCC_GENCODE="-gencode=arch=compute_121,code=sm_121"
cd ~
git clone https://github.com/NVIDIA/nccl-tests.git
cd nccl-tests
make MPI=1 MPI_HOME=/usr/lib/aarch64-linux-gnu/openmpi \
     CUDA_HOME=/usr/local/cuda NCCL_HOME=$HOME/nccl/build
ls ~/nccl/build/lib/libnccl.so.2 ~/nccl-tests/build/*_perf
```

The `NVCC_GENCODE` flag builds only for the GB10 GPU (compute capability 12.1), which cuts build time a lot; without it, `make -j` builds for every architecture.

5.4 Environment (dgx-01, before every NCCL run)

```
export LD_LIBRARY_PATH=$HOME/nccl/build/lib:/usr/local/cuda/lib64:/usr/lib/aarch64-linux-gnu/openmpi/lib
export UCX_NET_DEVICES=enp1s0f0np0
export NCCL_SOCKET_IFNAME=enp1s0f0np0
export OMPI_MCA_btl_tcp_if_include=enp1s0f0np0
export NCCL_IB_HCA=rocep1s0f0,roceP2p1s0f0
export NCCL_IB_GID_INDEX=3
export NCCL_IB_DISABLE=0
export NCCL_DEBUG=WARN
export NCCL_MIN_NCHANNELS=16
export NCCL_MAX_NCHANNELS=16
export NCCL_ALGO=Ring
```

Put this in a file such as `~/nccl-env.sh` and `source` it. Every variable a remote rank needs must also be passed with `-x` on the mpirun line.

5.5 PyTorch (on every node)

```
ALL NODES python3 -m venv ~/venvs/train && source ~/venvs/train/bin/activate
pip install --upgrade pip
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu130
python -c "import torch; print(torch.__version__, torch.version.cuda, torch.cuda.is_available(),
torch.cuda.get_device_name(0))"
```

```
2.12.1+cu130 13.0 True NVIDIA GB10
```

A warning that compute capability 12.1 is outside 8.0–12.0 is expected on GB10 and harmless. Keep the identical torch version on all nodes: mismatches hang collectives.

Exit check

SSH by fabric IP works from dgx-01 to all nodes; mpirun prints four hostnames; `libnccl.so.2` and the `*_perf` binaries exist on all four; torch reports `True NVIDIA GB10` on all four.

11 · Phase 6 · Validation ladder

Test bottom-up. Each rung depends only on the rungs below it, so the first failing rung tells you where to look.

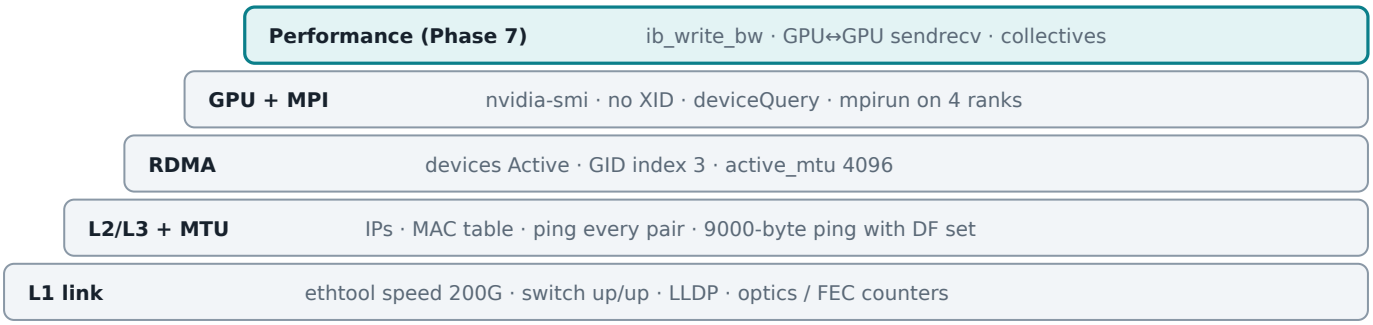


Figure 5. Validation ladder: start at the bottom.

6.1 Link layer

```
EACH NODE sudo ethtool enp1s0f0np0 | grep -E "Speed|Duplex|Link detected"
sudo ethtool -S enp1s0f0np0 | grep -E "rx_crc_errors_phy|rx_symbol_err_phy|link_down_events_phy"
```

Pass: 200000Mb/s, Full, yes; CRC and symbol errors 0 and not increasing. On the switch: `show interfaces et-0/0/0:0 extensive | match "errors|FEC"`: no input errors, uncorrected FEC 0.

6.2 Addressing, reachability and MTU

```
DGX-01 ip -4 -br a
for ip in 192.168.100.11 192.168.100.12 192.168.100.13 192.168.101.11 192.168.101.12 192.168.101.13; do
  ping -c 2 -W 1 $ip >/dev/null && echo "$ip ok" || echo "$ip FAIL"; done
ping -M do -s 8972 -c 3 192.168.100.13    # 8972 + 28 header = 9000 bytes, don't fragment
```

```
PING 192.168.100.13 (192.168.100.13) 8972(9000) bytes of data.
8980 bytes from 192.168.100.13: icmp_seq=1 ttl=64 time=0.302 ms
8980 bytes from 192.168.100.13: icmp_seq=2 ttl=64 time=0.291 ms
8980 bytes from 192.168.100.13: icmp_seq=3 ttl=64 time=0.288 ms
--- 192.168.100.13 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
```

Pass: every peer ok on both subnets; jumbo ping 0% loss to a node on the **other** switch, which proves MTU 9216 on the host, the leg, the ISL and the far leg. Small pings passing while jumbo pings fail means one hop has a smaller MTU.

6.3 RDMA devices, GIDs and RDMA MTU

```
EACH NODE rdma link
ibv_devices
show_gids | grep -E "rocep1s0f0|roceP2p1s0f0"
for d in rocep1s0f0 roceP2p1s0f0; do echo "== $d"; ibv_devinfo -d $d | grep -E "state|active_mtu|link_layer"; done
```

```
DEV             PORT  INDEX  GID                                     IPv4            VER  DEV
rocep1s0f0      1     2      0000:0000:0000:0000:0000:ffff:c0a8:640a  192.168.100.10  v1   enp1s0f0np0
rocep1s0f0      1     3      0000:0000:0000:0000:0000:ffff:c0a8:640a  192.168.100.10  v2   enp1s0f0np0
roceP2p1s0f0    1     2      0000:0000:0000:0000:0000:ffff:c0a8:650a  192.168.101.10  v1   enP2p1s0f0np0
roceP2p1s0f0    1     3      0000:0000:0000:0000:0000:ffff:c0a8:650a  192.168.101.10  v2   enP2p1s0f0np0
== rocep1s0f0
                        state:                  PORT_ACTIVE (4)
                        active_mtu:             4096 (5)
                        link_layer:             Ethernet
```

Pass: GID index 3 is v2 with the node's IPv4 on both devices; PORT_ACTIVE; active_mtu 4096. If active_mtu shows 1024 (3), the netdev MTU is still 1500: fix the netplan MTU (or `sudo ip link set <if> mtu 9216` for a quick, non-persistent test).

6.4 GPU health (single node)

```
EACH NODE nvidia-smi
nvidia-smi --query-gpu=name,driver_version,temperature.gpu,power.draw,pstate --format=csv
sudo dmesg | grep -i xid    # expect no output
```

Pass: `NVIDIA GB10` listed, driver and CUDA match the baseline, no XID lines. Memory-Usage shows "Not Supported" on GB10 because memory is unified; that is normal.

Optional GPU compute check with the CUDA samples (host ↔ device bandwidth and device properties):

```
git clone https://github.com/NVIDIA/cuda-samples.git && cd cuda-samples
mkdir build && cd build && cmake .. -DCMAKE_CUDA_ARCHITECTURES=121 && make -j deviceQuery bandwidthTest
./Samples/1_Utilities/deviceQuery/deviceQuery | grep -E "Device 0|CUDA Capability|Result"
```

Sample paths can differ by cuda-samples release; use `find . -name deviceQuery -type f` if needed. Expect "CUDA Capability Major/Minor version number: 12.1" and "Result = PASS".

Exit check
Every rung passes on every node. Record the results in Appendix B before starting Phase 7.

12 · Phase 7 · Performance tests: RDMA and GPU-to-GPU

7.1 Raw RDMA: one logical half, one pair

Start the server on the target, then the client. `-x 3` selects the RoCEv2 IPv4 GID; `-q 8` uses 8 queue pairs; `-s 1048576` sends 1 MiB messages.

```
DGX-04 ib_write_bw -d rocep1s0f0 -F --report_gbits -q 8 -s 1048576 -x 3 -p 18515
DGX-01 ib_write_bw -d rocep1s0f0 -F --report_gbits -q 8 -s 1048576 -x 3 -p 18515 192.168.100.13
```

```
 Mtu             : 4096[B]
 Link type       : Ethernet
 GID index       : 3
---------------------------------------------------------------------------------------
 #bytes     #iterations    BW peak[Gb/sec]    BW average[Gb/sec]   MsgRate[Mpps]
 1048576    40000          98.04              98.04                0.011687
```

Pass: about 98 Gb/s, the PCIe x4 ceiling of one half (repo result: 98.04 Gb/s).

7.2 Raw RDMA: both halves at once (whole 200G port)

Run two pairs in parallel, one per logical half, on different TCP ports. Their sum is the real per-Spark capacity.

```
DGX-04 ib_write_bw -d rocep1s0f0   -F --report_gbits -q 8 -s 1048576 -x 3 -p 18515 &
ib_write_bw -d roceP2p1s0f0 -F --report_gbits -q 8 -s 1048576 -x 3 -p 18516 &
DGX-01 ib_write_bw -d rocep1s0f0   -F --report_gbits -q 8 -s 1048576 -x 3 -p 18515 192.168.100.13 &
ib_write_bw -d roceP2p1s0f0 -F --report_gbits -q 8 -s 1048576 -x 3 -p 18516 192.168.101.13 &
wait
```

Pass: each near 98 Gb/s, about 195 Gb/s combined. If the sum is far below, check the switch leg speed (200G) and PFC pause counters.

7.3 Full mesh (every directed pair)

Use the repo's `Cluster-Setup/full_mesh_ib_wr.sh` from dgx-01. It runs 12 sequential pairs over SSH and saves `result_<srv>_<cli>.txt` files.

```
DGX-01 chmod +x full_mesh_ib_wr.sh && ./full_mesh_ib_wr.sh
grep -h "^ 1048576" result_*.txt
```

Pass: all 12 pairs within a couple of Gb/s of each other. One slow row or column points to one node's port or cable; pairs that cross the ISL can be slightly lower than same-switch pairs.

7.4 GPU memory to the NIC (optional)

If perftest was built with CUDA support, register GPU memory and move it straight over the fabric:

```
ib_write_bw -d rocep1s0f0 -F --report_gbits -x 3 --use_cuda=0              # on dgx-04
ib_write_bw -d rocep1s0f0 -F --report_gbits -x 3 --use_cuda=0 192.168.100.13  # on dgx-01
```

An "unrecognized option" error means the packaged perftest was built without CUDA; skip this test, since NCCL in 7.5 moves GPU buffers anyway. Results should be close to 7.1.

7.5 GPU-to-GPU point-to-point (NCCL sendrecv)

This sends data from GPU memory on one node to GPU memory on another: the most direct GPU↔GPU test the cluster has. Run it on a same-switch pair and on a cross-switch pair.

```
DGX-01 source ~/nccl-env.sh
mpirun -np 2 -H 192.168.100.10:1,192.168.100.13:1 \
  --mca plm_rsh_agent "ssh -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no -x" \
  -x LD_LIBRARY_PATH -x NCCL_SOCKET_IFNAME -x NCCL_IB_HCA -x NCCL_IB_GID_INDEX \
  -x OMPI_MCA_btl_tcp_if_include -x UCX_NET_DEVICES \
  $HOME/nccl-tests/build/sendrecv_perf -b 8 -e 4G -f 2 -g 1
```

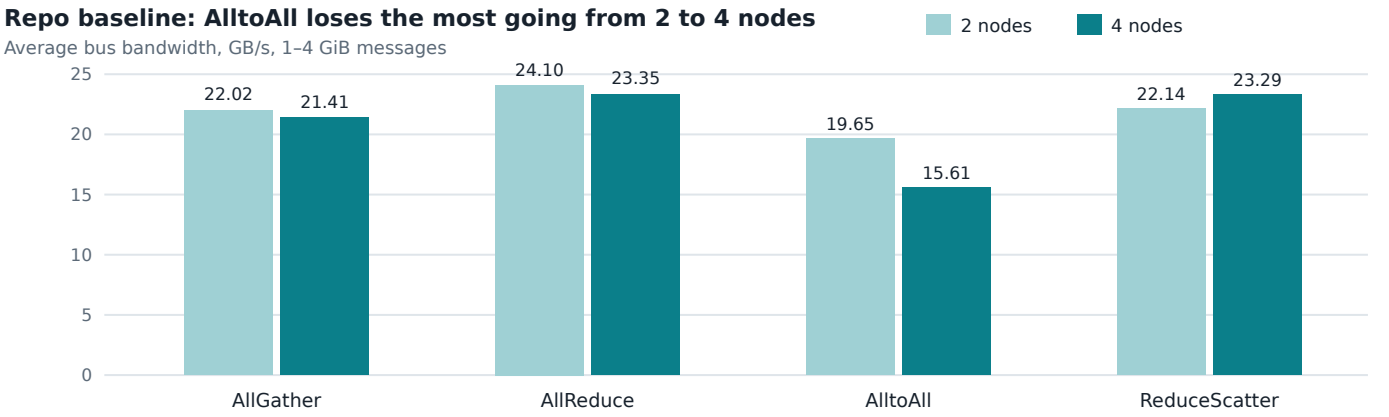
Pass: busbw climbs with message size and levels off in the low-20s GB/s range at large sizes (this is an estimate from the both-halves RDMA ceiling: record your own as the baseline). Small messages show latency, not bandwidth.

7.6 NCCL collectives on 2 and 4 nodes

```
DGX-01 mpirun -np 4 \
  -H 192.168.100.10:1,192.168.100.11:1,192.168.100.12:1,192.168.100.13:1 \
  --mca plm_rsh_agent "ssh -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no -x" \
  -x LD_LIBRARY_PATH -x UCX_NET_DEVICES -x NCCL_SOCKET_IFNAME -x NCCL_IB_HCA \
  -x NCCL_IB_GID_INDEX -x NCCL_MIN_NCHANNELS -x NCCL_MAX_NCHANNELS \
  -x OMPI_MCA_btl_tcp_if_include -x NCCL_ALGO \
  $HOME/nccl-tests/build/all_reduce_perf -b 1024M -e 4G -f 2 -g 1 -n 100
```

Repeat with `all_gather_perf`, `alltoall_perf` and `reduce_scatter_perf`, and with `-np 2 -H 192.168.100.10:1,192.168.100.11:1` for the 2-node runs. Read the last line, `# Avg bus bandwidth`.

```
#       size         count      type   redop    root     time   algbw   busbw #wrong
  1073741824     268435456     float     sum      -1    68812   15.60   23.40      0
  2147483648     536870912     float     sum      -1   137201   15.65   23.48      0
  4294967296    1073741824     float     sum      -1   272390   15.77   23.65      0
# Out of bounds values : 0 OK
# Avg bus bandwidth    : 23.35
```



Source: repo Cluster-Setup README, "Collective Summary". The repo's 2-node AllReduce and ReduceScatter rows ran other binaries (see Appendix C); re-measure those tw

Figure 6. Baseline collective performance from the repo testbed.

Collective	Repo 2-node avg	Repo 4-node avg	Your 2-node	Your 4-node	Why 4 differs from 2
AllGather	22.02	21.41			Ring; more hops, ISL now in path
AllReduce	24.10	23.35			Ring reduce + gather; scales well
AlltoAll	19.65	15.61			Every rank sends to every rank; most cross-switch traffic, most sensitive to congestion
ReduceScatter	22.14	23.29			Ring; 2-node repo value came from alltoall_perf

Pass: each result within about 10% of the baseline, `#wrong` 0 on every row, "Out of bounds values: 0 OK".

7.7 Prove the transport

Add `-x NCCL_DEBUG` and run once with `NCCL_DEBUG=INFO`:

```
NCCL_DEBUG=INFO mpirun ... -x NCCL_DEBUG ... all_reduce_perf -b 1G -e 1G -g 1 2>&1 | grep -E "NET/|Using
network|GDR"
```

```
nvidia-spark-dgx-01:41823:41823 [0] NCCL INFO NET/IB : Using [0]rocep1s0f0:1/RoCE [1]roceP2p1s0f0:1/RoCE [RO]; OOB
enp1s0f0np0:192.168.100.10<0>
nvidia-spark-dgx-01:41823:41823 [0] NCCL INFO Using network IB
```

Pass: `NET/IB` lists both P0 devices and "Using network IB". `NET/Socket` means NCCL fell back to TCP: look for `NCCL_IB_DISABLE=1` in the environment or `~/.bashrc`, or a wrong `NCCL_IB_HCA`.

7.8 Watch the traffic while a test runs

```
ANY NODE watch -n1 "ethtool -S enp1s0f0np0 | grep -E 'vport_rdma_unicast_bytes|prio3_pause|discards'"
SWITCH monitor interface traffic
show interfaces et-0/0/0:0 extensive | match "Drops|Priority"
```

Pass: RDMA byte counters climb on both P0 halves; with PFC on, a few pause frames and zero discards; on the switch, no output drops on Spark legs or ae0. Discards with zero pause frames means PFC isn't working end to end.

13 · Phase 8 · Workloads

8.1 Distributed sanity check (PyTorch all-reduce)

Copy the repo's `simple_training_job/dist_check.py` to the home directory on all four nodes. Each rank contributes its rank number and NCCL sums them: 0 + 1 + 2 + 3 = 6. Start the same command on every node in its own terminal, changing only `--node_rank`:

```
NODE N source ~/venvs/train/bin/activate
NCCL_DEBUG=WARN NCCL_IB_HCA=rocep1s0f0,roceP2p1s0f0 NCCL_IB_GID_INDEX=3 \
NCCL_SOCKET_IFNAME=enP7s7 \
torchrun --nnodes=4 --nproc_per_node=1 --node_rank=N \
  --master_addr=10.0.151.249 --master_port=29500 dist_check.py
```

```
NCCL version 2.29.7+cuda13.2
rank 0/4 all_reduce result = 6.0          # node 0; others print rank 1/4, 2/4, 3/4 ... = 6.0
```

Rendezvous and NCCL bootstrap use the management network (`--master_addr`, `NCCL_SOCKET_IFNAME=enP7s7`); the data itself moves over RoCE on `NCCL_IB_HCA`. If a run hangs, stop it everywhere with `pkill -9 -f torchrun; pkill -9 -f dist_check` and start all four again.

8.2 Multi-node DDP training

Copy `simple_training_job/train_ddp_spark.py` to every node (for example `scp ~/train_ddp_spark.py regress@10.0.151.250:~/`), then launch exactly as in 8.1 with the script name changed.

```
NCCL version 2.29.7+cuda13.2
epoch 0 loss 2.3498
epoch 1 loss 2.4395
epoch 2 loss 2.3287
epoch 3 loss 2.4334
epoch 4 loss 2.5293
saved checkpoint to ~/ckpt.pt
```

The loss stays near 2.3–2.5 because the inputs and labels are random: the test proves the distributed plumbing (launch, gradient all-reduce after each backward pass, checkpoint), not learning. Inspect the checkpoint on node 0:

```
python -c "import torch; sd=torch.load('$HOME/ckpt.pt'); [print(k, tuple(v.shape)) for k,v in sd.items()]"
```

```
0.weight (4096, 1024)
0.bias (4096,)
2.weight (10, 4096)
2.bias (10,)
```

While it runs, watch `ethtool -S` on any node: RDMA byte counters on `enp1s0f0np0` and `enP2p1s0f0np0` should climb each step as gradients are all-reduced.

Exit check
All four ranks print 6.0; DDP completes 5 epochs with no NCCL warnings; the checkpoint has the four tensors above.

8.3 Next workloads

The repo's future-work list: real datasets with PyTorch DDP, FSDP, DeepSpeed and Megatron-LM training; vLLM multi-node serving and disaggregated inference. Re-run Phase 7 after any driver, NCCL or firmware change and compare against Appendix B.

14 · Troubleshooting

Symptom	Likely cause	Check	Fix
Spark leg down, or ethtool shows 100000Mb/s	Wrong breakout channelization, autoneg settled low, bad DAC	`ethtool enp1s0f0np0`; switch `show interfaces et-0/0/0:0`	Fix `number-of-sub-ports` / `speed 200g`; reseat or swap the DAC
Interface Up but no IPv4 / wrong IP	Netplan typo (repo DGX-04 block)	`ip -4 -br a`; `cat /etc/netplan/40-cx7.yaml`	Correct the file, `sudo netplan apply`
Nodes on one switch reach each other, not the other switch	ae0 down or VLAN 100 missing on the trunk	`show lacp interfaces ae0`; `show ethernet-switching table`	Bring ae0 up; add `vlan members spark-fabric` to ae0
Small pings pass, 9000-byte pings fail	MTU below 9216 on one hop	`ping -M do -s 8972` per pair; `show interfaces … | match MTU`	Set MTU 9216 on that switch port / ae0 / host
active_mtu 1024 (3)	Netdev MTU 1500	`ibv_devinfo -d … | grep active_mtu`	Netplan `mtu: 9216`, apply
`ib_write_bw`: "Couldn't connect"	Server not started, wrong IP/port, path down	Is the server waiting? `ping` the IP	Start the server first; same `-p` on both sides
`ib_write_bw` fails "status 12" / retry exceeded	RDMA frames dropped (MTU mismatch, link errors)	Jumbo ping; switch input errors	Fix MTU or the cable
mpirun "ORTE was unable to reliably start…"	SSH key missing or node unreachable	`ssh 192.168.100.1X hostname`	Re-run `discover-sparks` or `ssh-copy-id`
"libnccl.so.2: cannot open shared object file"	`LD_LIBRARY_PATH` not set or not passed	`echo $LD_LIBRARY_PATH`	`source ~/nccl-env.sh` and pass `-x LD_LIBRARY_PATH`
mpirun "unable to launch … executable"	nccl-tests not built on that node	`ls ~/nccl-tests/build` on the named node	Build on all nodes
NCCL busbw about 10× low	NCCL on TCP sockets	`NCCL_DEBUG=INFO` shows NET/Socket; `env | grep NCCL`; `~/.bashrc`	Remove `NCCL_IB_DISABLE=1`; set `NCCL_IB_HCA` and `NCCL_IB_GID_INDEX=3`
4-node runs slow, AlltoAll worst; switch drops	No PFC/ECN, congestion across the ISL	Switch output drops; host `prio3_pause` = 0, `rx_discards_phy` rising	Configure PFC/ECN on switches and hosts (matching priority)
Job fails "status=12 … transport retry counter exceeded"	Packets dropped on a path, usually MTU	Which peer IP is in the warning? Check its switch port	Fix MTU or link on that port
`nvidia-smi` "Unable to determine the device handle"	GPU fault (for example XID 79, fell off the bus)	`sudo dmesg | grep -i xid`	`nvidia-bug-report.sh`, then reboot; repeated → support case
torchrun waits forever	One node not started, different `--master_addr`/port, or mismatched torch versions	Same command on all 4? `pip show torch`	`pkill` everywhere, align versions, relaunch all four

Appendix A · Environment variable reference

Variable	Value in this lab	Purpose
`LD_LIBRARY_PATH`	`$HOME/nccl/build/lib:…`	Finds the NCCL you built, CUDA and Open MPI libraries
`NCCL_IB_HCA`	`rocep1s0f0,roceP2p1s0f0`	RDMA devices NCCL may use: both halves of P0
`NCCL_IB_GID_INDEX`	`3`	RoCEv2 GID carrying the IPv4 address
`NCCL_IB_DISABLE`	`0`	`1` forces TCP sockets (10× slower)
`NCCL_SOCKET_IFNAME`	`enp1s0f0np0` (MPI tests) / `enP7s7` (torchrun)	Interface for NCCL's bootstrap / out-of-band connection
`NCCL_IB_TC`	`106` (only with PFC/ECN)	Traffic class: DSCP 26 with ECN-capable bits
`NCCL_MIN/MAX_NCHANNELS`	`16` / `16`	Fixes channel count for repeatable results (repo setting)
`NCCL_ALGO`	`Ring`	Fixes the algorithm for repeatable results; Tree is the alternative
`NCCL_DEBUG`	`WARN` / `INFO`	INFO shows transport, devices, channels
`UCX_NET_DEVICES`, `OMPI_MCA_btl_tcp_if_include`	`enp1s0f0np0`	Keeps Open MPI's own traffic on the fabric interface

Appendix B · Results record sheet

Test	Expected	Measured	Pass?	Notes
Link speed, all 4 nodes	200000Mb/s			
Jumbo ping, cross-switch pair	0% loss			
active_mtu, both P0 devices	4096			
ib_write_bw, one half	~98 Gb/s			
ib_write_bw, both halves combined	~195 Gb/s			
Full mesh, slowest pair	≥ 96 Gb/s			
sendrecv_perf, same switch, 4 GiB	record baseline			
sendrecv_perf, cross switch, 4 GiB	record baseline			
AllGather 2 / 4 nodes	22.02 / 21.41			
AllReduce 2 / 4 nodes	24.10 / 23.35			
AlltoAll 2 / 4 nodes	19.65 / 15.61			
ReduceScatter 2 / 4 nodes	re-measure / 23.29			
NCCL transport	NET/IB, 2 devices			
dist_check, 4 ranks	6.0 on all			
DDP training	5 epochs + ckpt			

Appendix C · Corrections to the repo README

1. DGX-04 netplan uses `enP2p1s0f1np1` / `192.168.101.23`; should be `enP2p1s0f0np0` / `192.168.101.13`.
2. "AllReduce 2 Nodes" runs `all_gather_perf`; "ReduceScatter 2 Nodes" runs `alltoall_perf`. Re-measure both.
3. The RDMA MTU note says ">4096"; it means below 4096. A 1500-byte netdev gives `active_mtu 1024 (3)`.
4. The `ib_write_bw` server and client are both labelled DGX-04.

5. `NCCL_IB_GID_INDEX=3` is described but not exported in the Stable Configuration.
6. nccl-tests is cloned inside `~/nccl` but run from `$HOME/nccl-tests`.
7. The training guide's `NCCL_IB_HCA=rocep1s0f1` points at Port 1, which has no addresses in this design; use `rocep1s0f0,roceP2p1s0f0`.
8. The testbed line "Dual 200G Ports per Node" and the physical diagram's two links per node: in this design one P0 cable carries both logical halves (section 2). State which ports are cabled.

Appendix D · Sign-off checklist

☐	Cutsheet complete; every cable labelled and audited
☐	Switches: ae0 up, LLDP peers correct, VLAN 100 and MTU 9216 on all fabric ports, PFC configured
☐	All nodes on one software baseline (OS, driver, CUDA, NIC firmware, torch)
☐	Both P0 halves Up at 200G with correct IPs on every node; Port 1 unused everywhere
☐	Passwordless SSH and 4-rank mpirun from dgx-01
☐	Validation ladder passed on every node (Appendix B rows 1–3)
☐	RDMA and GPU↔GPU baselines recorded; NCCL within 10% of baseline over NET/IB
☐	dist_check = 6.0 on 4 ranks; DDP training completed with checkpoint
☐	Results and any deviations committed to the repo

Sources: repo `dgx_cluster_training_job_lab` (Cluster-Setup and simple_training_job READMEs); NVIDIA DGX Spark playbooks "Connect Multiple DGX Spark through a Switch" (build.nvidia.com/spark/multi-sparks-through-switch) and "Connect two Sparks". Switch configuration, management addresses, QoS values and estimated test targets are examples to adapt.